

AA

AF

11

22

33

44

66

77

88

99

10

20

55

BA

Frame Aligner Verification

Specification-driven verification of a byte-stream frame aligner

8 DUT defects, 3 critical / high

Corrected RTL: 0 spec violations

100 % functional coverage

Results at a glance

8

DUT defects

3 critical / high; DUT-02 and DUT-03 had been filed as "spec gaps"

0

unexplained mismatches

every one of 253 spec violations on the delivered RTL is attributed to a known defect

100 %

functional coverage

14 spec-feature coverpoints, every cover property hit

910 k

fuzzed cycles cross-checked

corrected RTL = both Python spec models and the SV model: 0 differences

Delivered RTL: TEST FAILED, 253 spec violations and 222 of 3,605 test-plan checkpoints failed, 0 unexplained. **Corrected RTL:** TEST PASSED, 0 violations, all checkpoints, 0 assertion failures.

0x02

The design under test

One byte per clock; a frame is a 16-bit header (LSB first) and 10 payload bytes, back to back

Frame format and fr_byte_position



HEAD_1 = 0xAFAA (AA then AF)

HEAD_2 = 0xBA55 (55 then BA)

Align

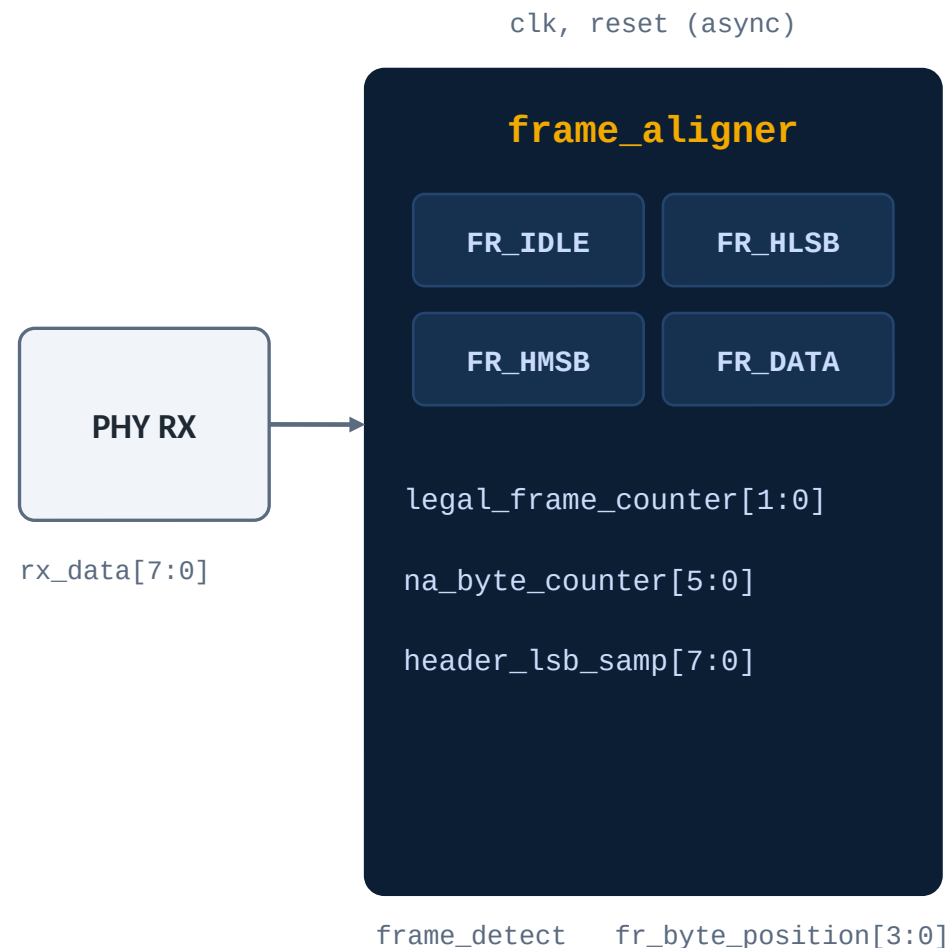
after 3 consecutive frames with a correct header: frame_detect = 1

Lose

after 4 frames without a header (48 header-less bytes): frame_detect = 0

Track

fr_byte_position = index 0..11 of the last byte (registered output)



From the specification to seven checkable rules

The text, the design slides and the RTL disagree in places; the verification fixes one interpretation

R1

Header = LSB immediately followed by its MSB: AA AF or 55 BA

text p.3, p.5

R2

While hunting, every byte is examined; a wrong MSB that is itself an LSB starts a new candidate. No header in the stream is missed

text p.3

R3

After a header the next 10 bytes are payload and are not examined

design slides (FSM)

R4

frame_detect rises one byte after the 3rd consecutive header (payload byte 0)

text p.6, waveform 1

R5

frame_detect falls on the 48th counted hunting byte (a header LSB counts when it arrives); header MSB and payload never count

text p.6, register table

R6

fr_byte_position = index of the byte just consumed (0..11); 0 while hunting

port table, waveform 1

R7

Asynchronous reset: outputs 0, hunting restarts

port table

Implemented three times, independently: SV reference model, a causal Python model and an offline frame parser. They agree on every cycle.

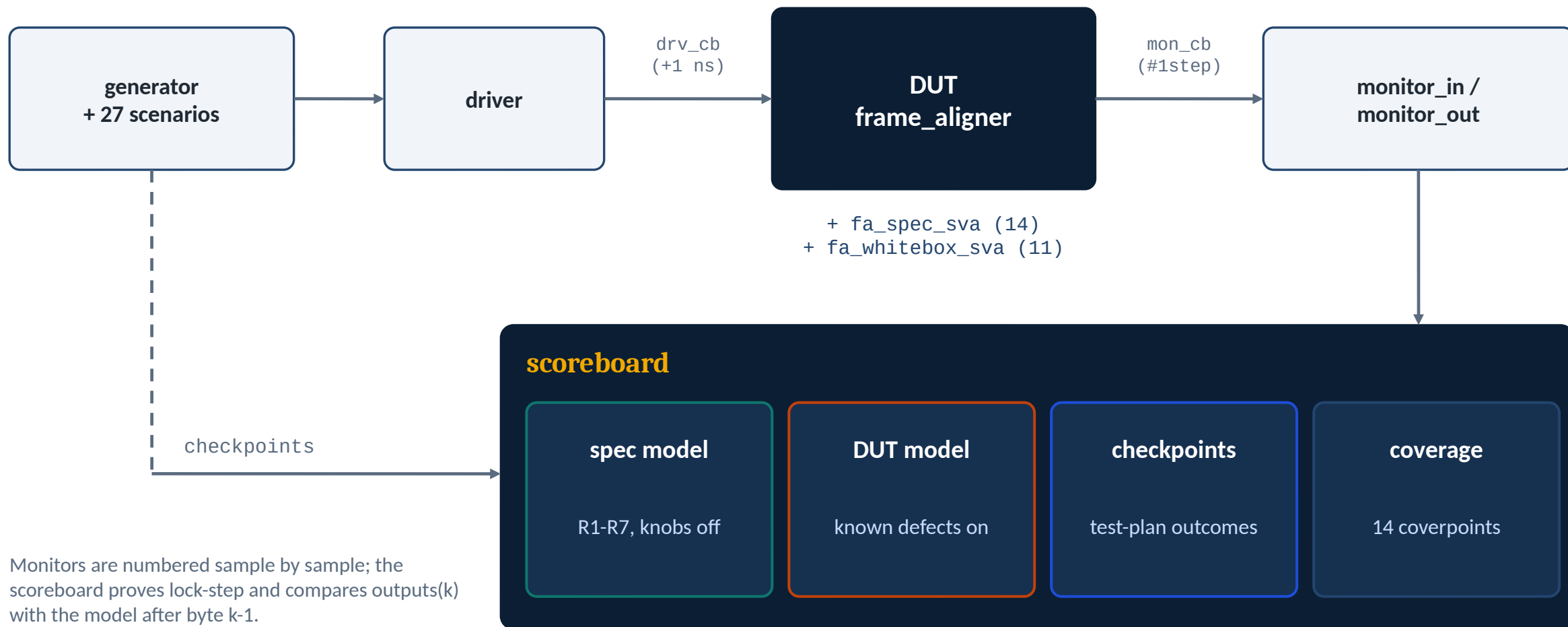
Why the environment was rebuilt

The original bench could not fail: its model was a copy of the RTL, and mismatches were fixed in the scoreboard

	Original environment	New environment
Reference	copy of the RTL (and even that one byte early)	spec rules R1-R7, cross-checked by two Python models
Mismatch policy	"correct the scoreboard"	attribute to a known defect, or flag as UNEXPLAINED
Expected outcomes	not checked	3,605 checkpoints at exact byte positions
Stimulus	X bytes; 117 of 917 items never driven; preconditions shuffled away	2-state; everything driven; every scenario starts from reset
Timing	program-block dependent; reset released on a clock edge	clocking blocks; reset on the falling edge
Assertions	3 of 3 vacuous or false; failures not counted	26 rule-based assertions, counted; cover hits show they are exercised
Coverage	reachable illegal bins, unreachable bins, no spec features	14 spec-feature coverpoints, closed at 100 %

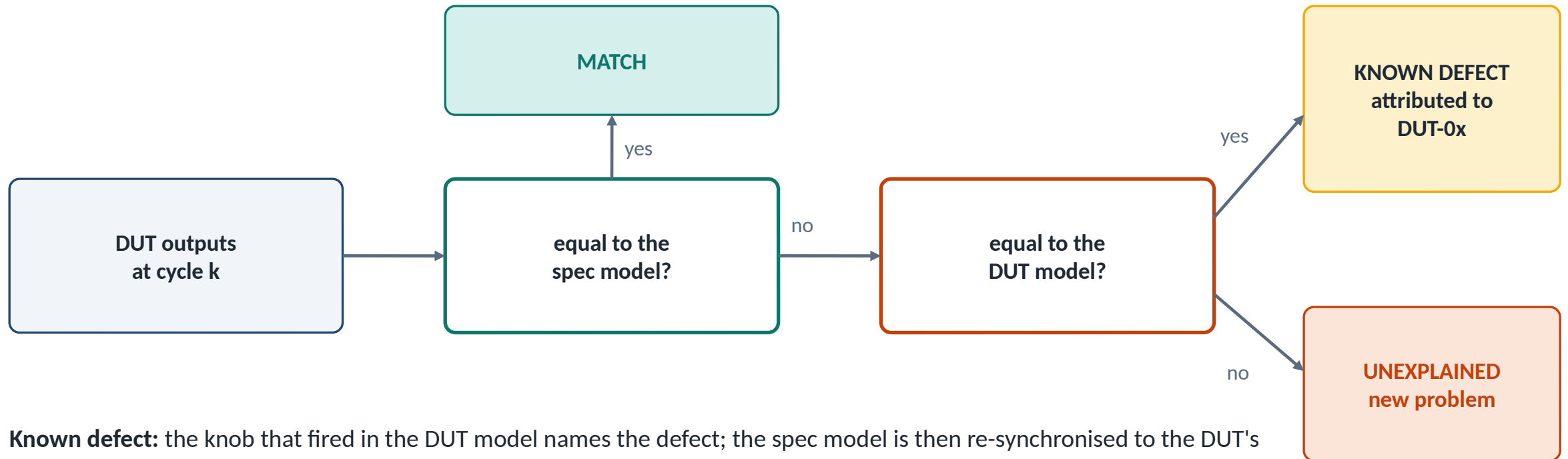
Verification architecture

Class-based SystemVerilog, race-free clocking blocks, assertions bound into the DUT



Every mismatch is triaged, not "fixed"

Two instances of the same specification model: one pure, one with the delivered design's defects switched on



Known defect: the knob that fired in the DUT model names the defect; the spec model is then re-synchronised to the DUT's path, so each occurrence is reported once, with no cascade.

+MODEL=dut: the same bench becomes a regression bench for the delivered RTL and fails only on NEW behaviour.

Result on the delivered RTL: 253 spec violations, 0 unexplained.

Three independent oracles, three independent models

1 Reference model

every byte, every cycle

cycle-accurate compare of frame_detect and fr_byte_position against R1-R7

2 Checkpoints

3,605 expected outcomes

the test plan's "expected outcome" column, checked at the exact byte (e.g. "aligned after 47 header-less bytes, lost on the 48th")

3 Assertions

15 black-box + 11 white-box

spec rules on the ports; FSM and counter checks inside the DUT catch latent defects the ports never show

Differential fuzzing: 910 k cycles (Python models), 452 k replayed in the SV bench

corrected RTL = Python causal model = independent frame parser = SV spec model: **0 differences**

delivered RTL = bug-emulating model: **0 differences** (every deviation is DUT-01, 02 or 03)

delivered RTL vs. specification: **13 % of all cycles wrong** on stress traffic

Test plan (1/2)

Every scenario starts from reset and checks its expected outcome at exact bytes (checkpoints)

ID	+TEST=	Stimulus	Expected (spec)	Exposes
TP01	spec_suggested	5xHEAD_1, 4 illegal, 5xHEAD_2 (spec p.23)	aligned on frame 3; lost on the 48th byte	
TP02	mixed_headers	HEAD_1 / HEAD_2 in any mix	aligned after 3 frames	
TP03	swapped_header	AF AA / BA 55; 4 of them while aligned	never aligns; loss on the 48th byte	DUT-03
TP04	inverted_header	bit-inverted and bit-reversed headers	never aligns	DUT-03
TP05	lsb_ok_msb_bad	AA 01, 55 01, AA 00, 55 00, AA BA, 55 AF	position 0 after the rejected MSB	DUT-03
TP06	lsb_bad_msb_ok	0A AF, 05 BA	position 0, never aligns	
TP07	msb_in_payload	AF / BA bytes inside payloads	frame boundaries unchanged	
TP08	header_in_payload	headers at every payload offset	ignored (R3)	
TP09	restart_header	AA AA AF, 55 55 BA, AA 55 BA, 55 AA AF, ...	header found; aligned after 3 frames	DUT-01
TP10	loss_boundary	aligned, 44..48 header-less bytes, header	kept for <= 46, lost for >= 47	DUT-02
TP11	corrupted_frames	aligned, 1..4 corrupted frames	kept for <= 3, lost on byte 48 for 4	DUT-03
TP12	header_in_illegal	legacy illegal frames with inner headers	frame at the inner header	
TP13	frames_in_illegal	3 frames inside a 49-byte illegal frame	aligned exactly on byte 29	
TP14	header_soup	49-byte frames of random header bytes	model-checked	DUT-01/03

Test plan (2/2)

Every scenario starts from reset and checks its expected outcome at exact bytes (checkpoints)

ID	+TEST=	Stimulus	Expected (spec)	Exposes
TP15	lsb_at_payload_end	payload ends in AA / 55 / AA AF (aligned)	next header found	
TP16	long_valid_run	10 consecutive frames	aligned; counter saturates	DUT-04
TP17	long_garbage	80 header-less bytes, re-align	lost on 48; counter saturates	DUT-05
TP18	reset_every_phase	async reset in each phase, aligned or not	outputs 0; no header across reset	
TP19	false_lock_in_sync	corrupted header + header pattern in payload	locks on the false frame	DUT-07 demo
TP20	consecutive_rule	V, 1 byte, V, V, V; V, V, stray LSB, V, V, V	aligned on the 3rd frame after the break	
TP21	header_across_items	... AA AF ... across two items	recognised	
TP22	illegal_lengths	2 + 12 + 36 and 49 header-less bytes (aligned)	lost on exactly the 48th byte	
TP23	boundary	sweep 0..60 header-less bytes, +/- stray LSB	kept iff gap + prefix <= 46	DUT-01/02
TP24	loss_cause	48th byte = rejected MSB / restart LSB	lost on that byte	DUT-01
TP25	midstream_entry	start mid-stream; payloads end in one 55 / AA	aligned after 3 frames	DUT-01
TP26	error_then_lsb_payloads	1 header error, then payloads ending in 55	alignment kept	DUT-01
TP27	slip_in_sync	13-byte frames before / after alignment	hunting keeps alignment	DUT-07 demo
RND	random	60 % valid, 18 % illegal, 8 % restart, 12 % gap, 2 % reset	model + assertions	all

Spec-feature coverage and assertion results

Coverage is sampled from the reference model, so it measures behaviour, not raw signal values

CP02 / CP03

every transition (hunt, LSB, reject, restart, frame) x aligned

CP06

every byte that can follow a header LSB

CP07

gap before a header while aligned: 0 .. 44, 45, 46 (last chance)

CP08

every kind of 48th byte: plain, LSB, rejected MSB, restart LSB

CP09

fr_byte_position 0..11 x frame_detect

CP11

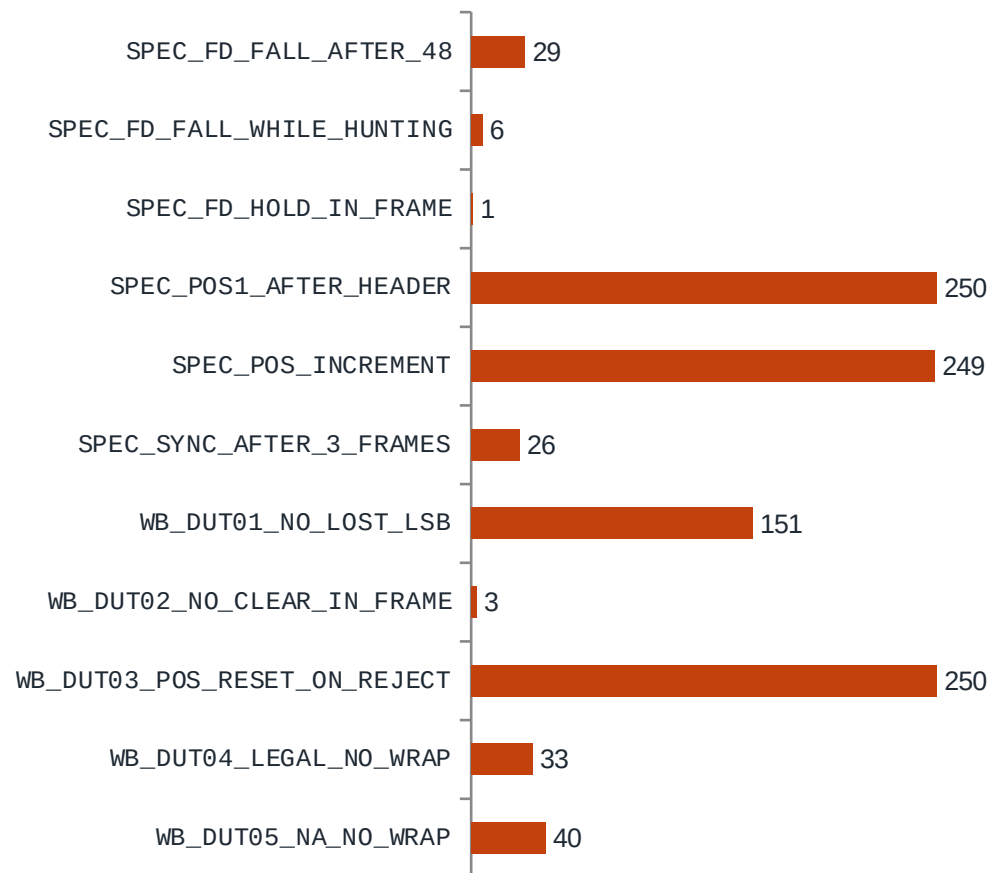
asynchronous reset in every phase, aligned or not

CP12

header patterns inside payloads

100 % of 14 coverpoints (102 bins) in the regression; every cover property hit

Assertion failures on the delivered RTL (corrected RTL: 0)



DUT defects

The student's scoreboard mirrored the RTL, so these were either unseen or filed as "gaps in the spec".

DUT-01

Header LSB lost: some legal streams never align

Critical

DUT-02

Alignment dropped on a valid header

High

DUT-07

No fly-wheel: slips never lose alignment

High (arch.)

DUT-03

Position 1 reported for a rejected header

Medium

DUT-04

Frame counter wraps 3 -> 0

Low (latent)

DUT-05

Byte counter wraps 63 -> 0

Low (latent)

DUT-08

X-optimistic header decode

Low

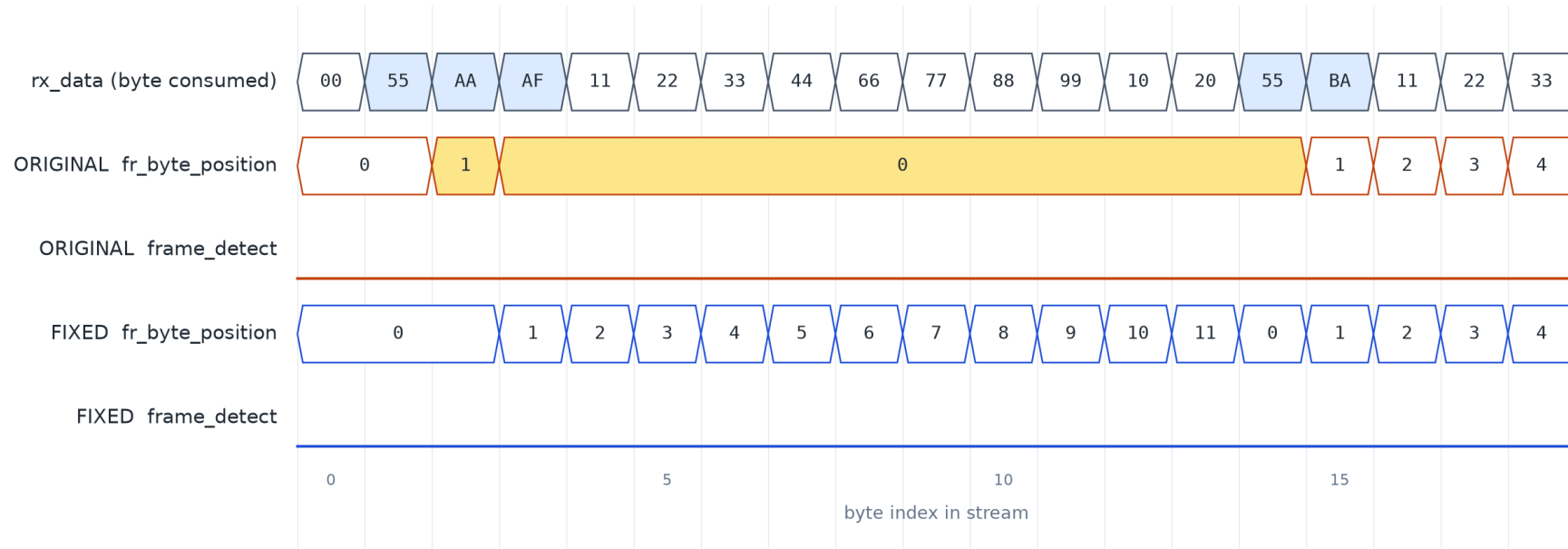
DUT-06

Coding issues (default, width, comments)

Info

Header LSB thrown away after a rejected MSB

Critical · extends the student's "LSB twice then MSB": it happens for all four LSB combinations



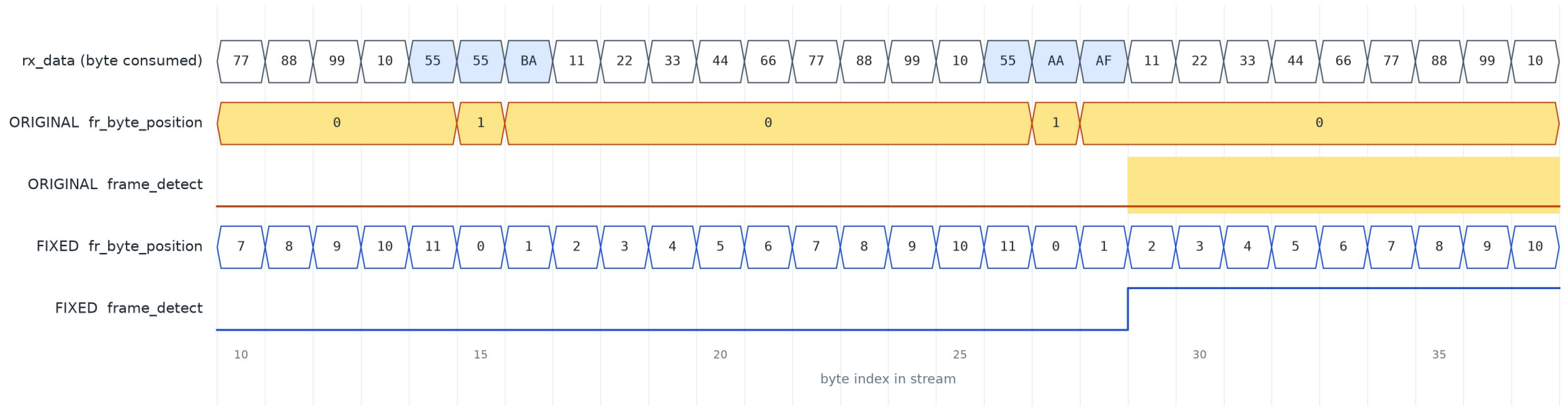
Root cause

In FR_HLSB a wrong MSB always returns to FR_IDLE. If that byte is itself a header LSB (AA / 55) it is never examined, so its header is missed.

```
end else if (header_lsb_valid) begin // FIX DUT-01
    legal_frame_counter_rst = 1'b1; fr_byte_position_rst = 1'b1;
    na_byte_count_inc       = 1'b1; next_state = FR_HLSB;
end
```

Some legal streams are never aligned

A single 0x55 (or 0xAA) as the last payload byte looks like a header LSB; the real LSB rejects it and is lost, in every frame



never

aligned: start-up inside a legal stream whose payloads end in a single 0x55 (TP25: 20 of 20 checkpoints fail)

permanent

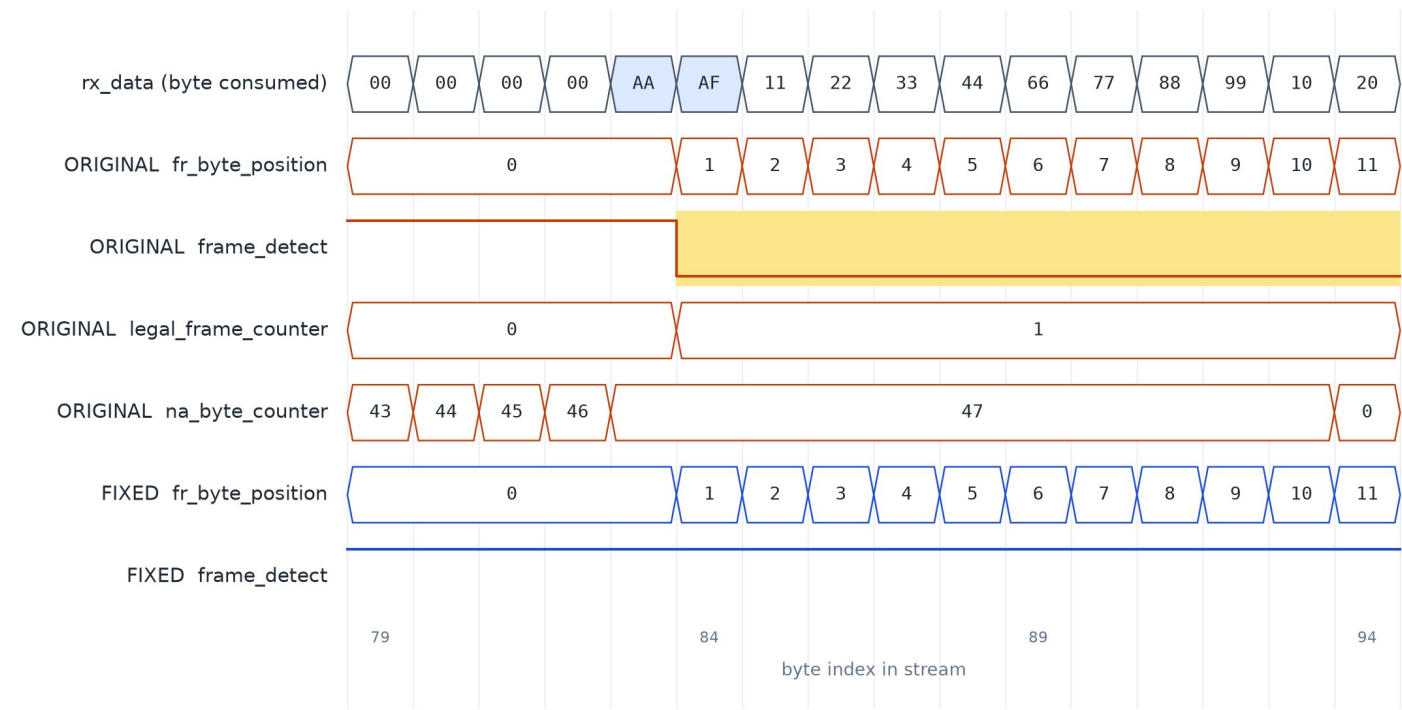
loss after one bit error in a header, if the following payloads end in 0x55 (TP26)

1 in 128

chance that only 3 bad frames already lose alignment: the byte before the 4th header is 0xAA / 0x55

Alignment dropped on a valid header

High · the student saw "last chance is a header at byte 45" and filed it as a spec gap



Root cause

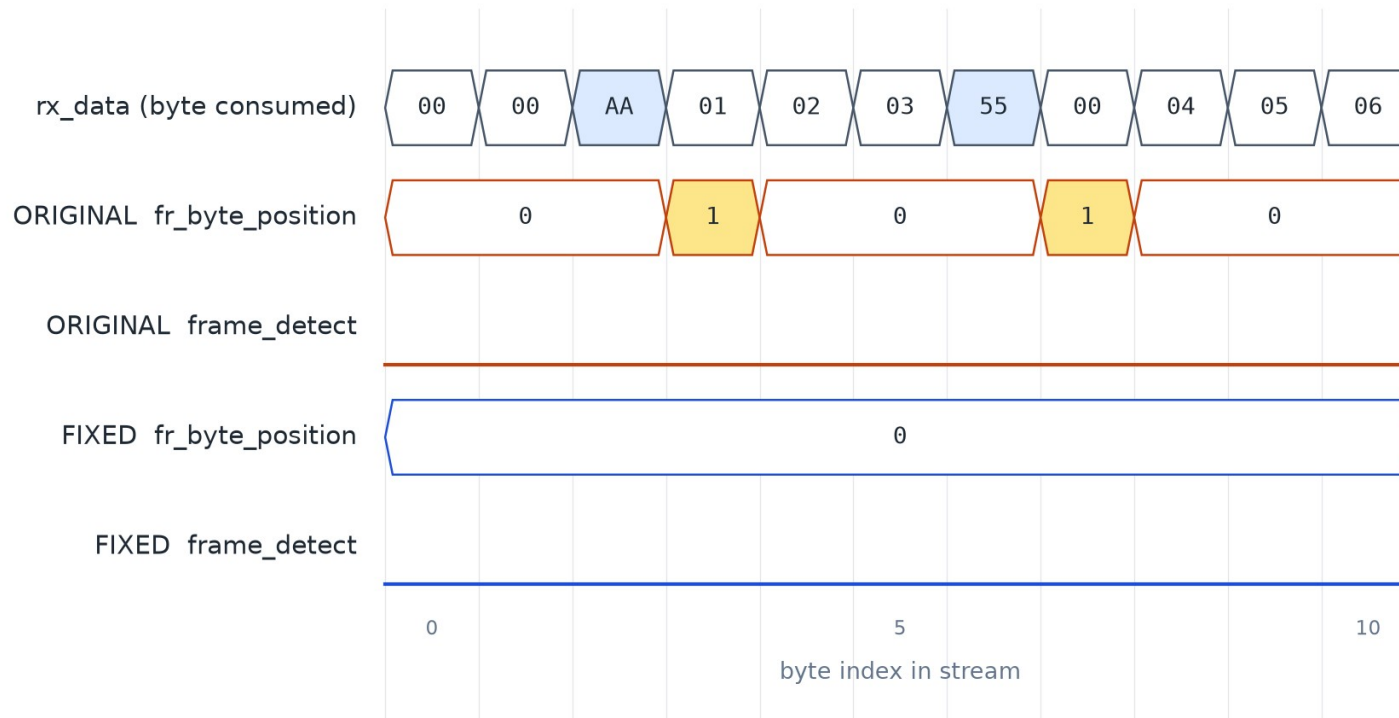
- after 46 header-less bytes the header LSB brings the counter to 47 (correct: it reaches 48 only on the next counted byte)
- but the clear (na_byte_counter == 47) is not qualified by na_byte_count_inc: the valid MSB clears frame_detect
- the whole valid frame is received with frame_detect = 0; outage 25 cycles

gap	spec	delivered RTL
<= 45	kept	kept
46	kept	LOST
>= 47	lost	lost

Fix: clear only when the consumed byte is itself counted: `na_byte_count_inc && na_byte_counter == 47`

Position reported for a rejected header

Medium · the student noted the one-cycle "1" and filed it as a spec gap



What happens

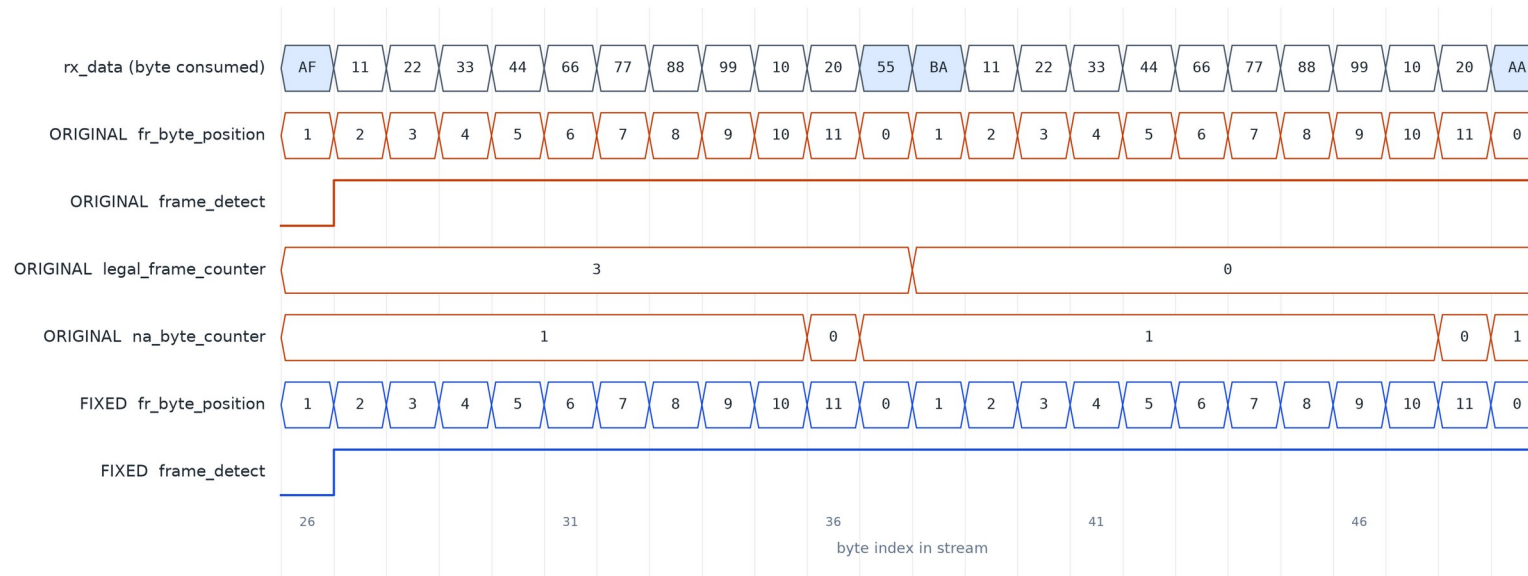
- after AA 01 the output says "header MSB" (position 1) although no frame exists
- every other hunting byte reports 0, so a consumer sees a phantom header
- the FSM slide omits the position reset on the HLSB -> IDLE arc too

Fix: `fr_byte_position_rst = 1` on the reject arc. Caught by `SPEC_POS1_AFTER_HEADER`, `SPEC_POS_INCREMENT` and `WB_DUT03_POS_RESET_ON_REJECT`.

DUT-04

Latent and hygiene defects

Invisible on the ports today; caught at the root cause by white-box assertions and lint

**DUT-04**

legal_frame_counter wraps 3 -> 0 on the 4th frame (above). Caught by WB_DUT04_LEGAL_NO_WRAP. Fix: saturate.

DUT-05

na_byte_counter wraps 63 -> 0 on long header-less streams. Caught by WB_DUT05_NA_NO_WRAP. Fix: saturate.

DUT-06

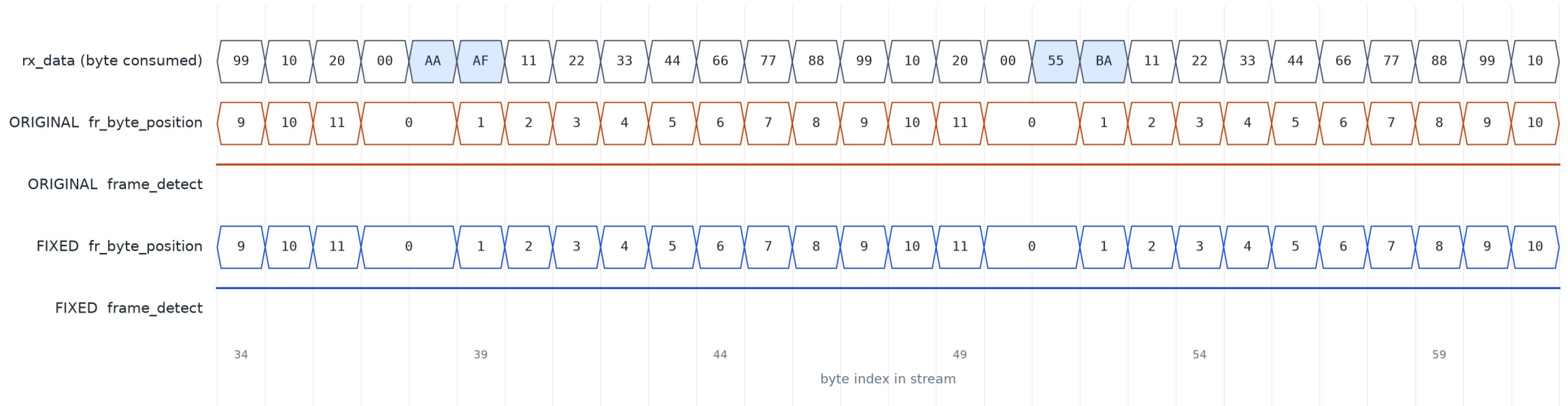
no next_state default; 4-bit vs 8'd10 compare; wrong comments. The corrected RTL is verilator -Wall clean.

DUT-08

X on rx_data is silently decoded as "not a header" in RTL, but not in gates. TB_IN_KNOWN forbids X stimulus.

No fly-wheel: slips never lose alignment

High (architecture) · spec text: out of alignment after four frames without the EXPECTED header



13-byte frames: every header one byte late, and frame_detect stays 1 forever. The same stream never aligns from reset (hysteresis).

Tracking stops: after one bad header, fr_byte_position is 0 for a whole frame while frame_detect = 1; a header pattern in a payload re-bases the frame (false lock).

Proposed fix: while aligned, count 0..11 freely, check headers only at the expected position, and declare out-of-frame after 4 bad frames. Hunt byte by byte only while not aligned.

Issues in the specification itself

To be clarified with the designer; each affects what "correct" means

SPEC - 01

Output data interface (header + payload forwarding) is specified but not implemented

SPEC - 02

Text: loss after 4 frames without the expected header. Design slides: 48 hunting bytes. Leads to DUT-07

SPEC - 03

Waveform 2 is one clock late against the FSM slide and the RTL; waveform 1 starts in an unreachable state

SPEC - 04

fr_byte_position is undefined outside a frame and has no valid qualifier (0 = LSB and 0 = not aligned)

SPEC - 05

Name mismatches: fr_is_aligned / frame_detect, na_frame_counter / na_byte_counter, Valid_h2 missing

SPEC - 06

The 48th header-less byte is a header LSB: counted (register table) or not (look-ahead)? The RTL violates both

Defects in the original testbench

Why the original run looked clean although the design is not

12.8 %

of the queued stimulus never driven (117 of 917 items): end of test compared transactions with repeat_count

1 byte

early: the "copied" model sets and clears frame_detect one byte before the RTL, so no clean run was possible

102

false assertion failures per run from one legacy property; the other two are vacuous or always fail

TB-01

model copied from the RTL; mismatches fixed in the scoreboard

TB-04

5 directed tests drive X (new[N] on a 4-state array, 8'hxA)

TB-05

tests shuffled: preconditions not met, tests spill into the next one

TB-07

coverage: reachable illegal bins, unreachable bins, no spec features

TB-08

sampling depends on program-block scheduling; reset on a clock edge

TB-09

2-state scoreboard turns X/Z into 0

TB-10

build order wrong; bind by instance name not elaborated

TB-11

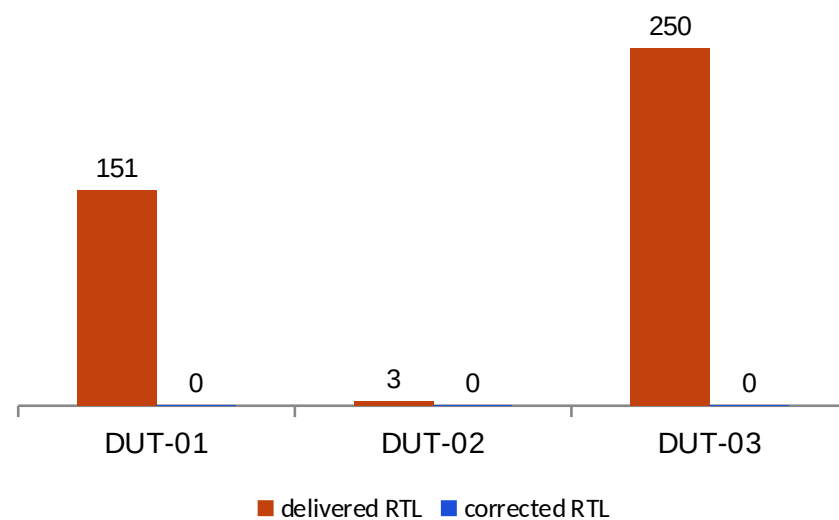
out-of-bounds writes, misleading "illegal" frames, no seed control

Regression results

make -C sim regress: every run is checked against its EXPECTED outcome

RTL	test	model	cycles	checkpoints	verdict	expected
corrected	directed	spec	4,424	1,045 / 1,045	PASS	PASS
corrected	boundary	spec	9,701	2,560 / 2,560	PASS	PASS
corrected	random	spec	6,485	-	PASS	PASS
corrected	regression	spec	21,079	3,605 / 3,605	PASS	PASS, cov 100.0 %
delivered	regression	DUT	21,079	tolerated	PASS	PASS (nothing new)
delivered	regression	spec	21,079	3,383 / 3,605	FAIL	FAIL, 0 unexplained
corrected	fuzz replay	spec	452,212	-	PASS	PASS
delivered	fuzz replay	spec	452,212	-	FAIL	FAIL, 0 unexplained

Spec violations attributed per defect



- RTL untouched: hash of the annotated DUT = hash of the delivered dut.sv
- The corrected RTL is verilator -Wall clean
- Python fuzzing, 910 k cycles: corrected RTL = both Python spec models; delivered RTL = bug-emulating model (0 differences)

Checking the checkers: mutation testing

One bug at a time is injected into the corrected RTL; the bench must detect every one (scripts/mutation_test.py)

20 / 20

mutants killed

- each mutant re-runs the directed and boundary tests
- killed = at least one oracle fails: scoreboard (SB), checkpoints (CP), spec SVA, white-box SVA (WB)
- M20 survived the first run (no stray LSB between frames): TP20 extended
- M07 / M14 escaped every black-box assertion (only an early loss was checked):
SPEC_FD_LOST_AFTER_48 added
- M04, M05, M19 are invisible on the ports: only white-box assertions can catch them

ID	injected bug	SB	CP	SVA	WB
M01	restart branch returns to IDLE (re-introduces DUT-01)	100	201	22	114
M02	unqualified sync-loss clear (re-introduces DUT-02)	13	12	6	3
M03	no position reset on reject (re-introduces DUT-03)	59	10	121	59
M04	legal counter wraps again (re-introduces DUT-04, latent)				17
M05	not-aligned counter wraps again (re-introduces DUT-05, latent)				5
M06	loss threshold 47 bytes instead of 48	55	23	42	
M07	loss threshold 49 bytes instead of 48	95	23	39	
M08	frame of 13 bytes (payload end one byte late)	9811	1244	993	537
M09	frame of 11 bytes (payload end one byte early)	8111	1537	963	
M10	alignment after 2 frames instead of 3	2372	530	177	
M11	HEAD_2 MSB constant wrong (0xBB)	8890	1407	176	
M12	only HEAD_1 LSB recognised	8890	1407	176	
M13	expected MSBs swapped between the header types	12236	2233	238	
M14	alignment never lost (clear removed)	727	156	274	
M15	not-aligned counter not cleared at the end of a frame	312	65	50	
M16	consecutive-frame counter not cleared on a hunting byte	1031	154	275	
M17	fr_byte_position reset value 1	252		616	
M18	header_lsb_samp never updated after reset	12145	2233	223	
M19	FR_HMSB skipped (equivalent on the ports; FSM structure only)				746
M20	restart does not reset the consecutive-frame counter	48	12	2	

Conclusions

The design is not bug-free. Its header search can miss every header of a legal stream (DUT-01) and drops alignment on a valid header (DUT-02).

A scoreboard that copies the RTL cannot find design bugs. The spec model with defect triage found all of them, with 0 unexplained mismatches.

Expected outcomes must be executable. 3,605 checkpoints turn the test plan into checks.

Evidence, not assumption. Three independent models agree; the corrected RTL passes everything; 20 of 20 injected bugs are caught.

Next steps

- Designer to accept DUT-01..06 fixes (rtl/frame_aligner_fixed.sv)
- Decide on the fly-wheel (DUT-07) and resolve SPEC-02/-06
- Add the output data interface (SPEC-01)
- Run the bench on a commercial simulator to merge native covergroups
- Formal proof of R1-R7 on the corrected RTL

Appendix: how to run

Verilator >= 5.030 with z3; Python 3; Icarus Verilog for scripts/

```
cd sim
make sim DUT=fixed           # corrected RTL           -> TEST PASSED
make sim DUT=orig           # delivered RTL           -> TEST FAILED, defects listed
make sim DUT=orig MODEL=dut # regression mode      -> only new behaviour fails
make sim DUT=orig TEST=restart_header VERBOSITY=2
make sim DUT=orig TEST=loss_boundary WAVES=1 # waves.vcd
make regress                # full matrix, expected outcomes
python3 ../scripts/fuzz_rtl.py # differential fuzzing (Icarus)
python3 ../scripts/plot_waves.py # regenerate the figures
python3 ../scripts/mutation_test.py # mutation score of the checkers
```

Documents: docs/BUG_REPORT.md (every defect, reproduction, fix) · docs/VERIFICATION_PLAN.md (rules, test plan, coverage, assertions, results)